🎵 Festava Live — GSAP Animation Project

> A fully animated music festival website built to demonstrate advanced GSAP (GreenSock Animation Platform) techniques. Created for a YouTube playlist teaching GSAP animations to students and developers.

GSAP 3.13.0

HTML5

CSS3

JAVASCRIPT ES6+

📋 Table of Contents

- 🎵 Festava Live — GSAP Animation Project
  - 📋 Table of Contents
  - 🎯 Project Overview
    - What makes this project special?
  - 📁 File Structure
  - 🛠️ Technology Stack
  - 🏗️ HTML Structure
  - 🎨 CSS Architecture
    - z-index Layers
    - Key CSS Techniques
  - ⚡ JavaScript Functions
    - 6.1 `cursorAnimation()`
    - 6.2 `animateNavbar()`
    - 6.3 `animateHeroHeading()`
    - 6.4 `animateAbout()`
    - 6.5 `animateAboutImage()`
    - 6.6 `animateArtist()`
    - 6.7 `animatePlans()`
    - 6.8 `animateGallery()`
    - 6.9 `animateTextSection()`
    - 6.10 `animateTextSection_SVG()`
    - 6.11 `animateFooter()`
  - 📚 GSAP Core Concepts
    - Easing Quick Reference
  - 📍 ScrollTrigger Reference
    - start and end Format
    - Common Examples
    - toggleActions
    - Debug Markers

- Common Issues & Fixes
- Future Improvements
  - Animation Enhancements
  - New Sections
  - Performance
  - Code Quality
- How to Run
- License

Project Overview

Festava Live is a complete music festival experience built entirely with vanilla HTML, CSS, and JavaScript — using GSAP as the animation engine.

What makes this project special?

Feature	Description
Custom Cursor	Changes shape on hover over different sections
Video Background	Full-screen autoplay video on hero section
Horizontal Gallery	Pinned scroll with GSAP ScrollTrigger
3D Tilt Cards	Artist cards with mousemove depth effect
Wave Text	Characters animate individually in wave pattern
SVG Blob	Follows cursor in the text section
Cursor Morph	Cursor becomes an image on about section hover
Footer Logo Hover	Reveals floating image on hover

File Structure

```
festava-live/
  ├── index.html                    # Main HTML — all sections
  ├── style.css                     # All CSS styles
  ├── script.js                     # All GSAP animations
  └── assets/
        ├── hero-section-video.mp4   # Hero background video
        ├── about-img.jpg            # About section background
        ├── about-img-2.jpg          # About section right image
        ├── artist-1.jpg             # Left big artist card
        ├── artist-2.jpg             # Right artist top
        ├── artist-3.jpg             # Right artist bottom
        └── music-artist-img.avif   # Cursor hover image
```

> All three files work together. HTML defines structure, CSS defines appearance, JS adds all animations. GSAP is loaded via CDN — no npm or build tools needed.

🛠 Technology Stack

Technology	Version	Purpose
HTML5	Latest	Structure and semantic markup
CSS3	Latest	Styling, layout, transitions
JavaScript	ES6+	Logic and DOM manipulation
GSAP	3.13.0	All animations (core engine)
ScrollTrigger	3.13.0	Scroll-based animation triggers
Font Awesome	6.5.2	Icons (social, arrows, etc.)

📐 HTML Structure

The entire website is one HTML file divided into sections. Each section is independently styled and animated.

Element	ID / Class	Purpose
`div`	`#cursor .cursor-box`	Custom animated cursor
`div`	`.video-container`	Full screen video background
`div`	`.overlay`	Dark overlay on top of video
`section`	`.main`	Contains navbar + hero
`nav`	`.navbar`	Navigation bar with links
`div`	`.hero`	Hero content (title, button, status)
`section`	`.about`	About section with two columns
`section`	`#artist-section`	Artist cards with images
`section`	`#plans-section`	Pricing cards
`section`	`.gallery-section`	Intro text + horizontal gallery
`section`	`#animation-text-section`	Wave text + SVG blob
`footer`	`#footer`	Footer with logo, nav, newsletter

> **Note:** The `#cursor` div is placed at the very top of `<body>` — before all sections. This ensures it always renders above everything else due to `z-index: 9999`.

🎨 CSS Architecture

z-index Layers

```
z-index: -2    →  Video background (.video-container)
z-index: -1    →  Dark overlay (.overlay)
z-index:  1    →  Main content (.main)
z-index:  2    →  Artist info overlay (.artist-info)
z-index:  9998 →  SVG blob (#blob)
z-index:  9999 →  Custom cursor (.cursor-box)
```

Key CSS Techniques

`position: fixed` vs `position: absolute`

- `fixed` — stays in place even when scrolling. Used for: cursor, overlay, video background
- `absolute` — positioned relative to nearest positioned parent. Used for: image overlays, about-card, artist-info

`will-change: transform`

Applied on elements that GSAP will animate. Tells the browser to prepare GPU acceleration in advance — smoother animations, fewer repaints.

`clamp()` for Responsive Font Sizes

```
font-size: clamp(3rem, 8vw, 7rem);
/*              ^min  ^fluid  ^max */
```

No media queries needed. Font scales automatically between min and max based on viewport width.

⚡ JavaScript Functions

All animations are organized into separate functions. Each function handles one section of the website. This makes the code modular — easy to debug, update, or teach one section at a time.

> **Pattern:** Every function is defined first, then called at the bottom of the file. If an animation breaks, you can comment out just that one function call to isolate it.

6.1 `cursorAnimation()`

What it does: Makes the orange circle `#cursor` div follow the mouse pointer across the entire page with a smooth lag effect.

```
document.addEventListener('mousemove', (e) => {
  gsap.to('#cursor', {
    x: e.clientX,      // horizontal position from left of viewport
    y: e.clientY,      // vertical position from top of viewport
    duration: 1,       // 1 second lag — creates trailing effect
    ease: 'power2.out' // fast start, slow end = natural feel
  });
});
```

Why `clientX` and not `pageX`?

Property	Relative to	Use case
`clientX`	Viewport (visible screen)	☑ Cursor (position: fixed)
`pageX`	Entire page including scroll	✗ Would jump on scroll
`screenX`	Monitor screen	✗ Unreliable on multi-monitor
`offsetX`	Element that was clicked	✗ Changes per element

CSS Requirements:

```
.cursor-box {
  position: fixed;                /* stays fixed while scrolling */
  top: 0; left: 0;                /* positioned from top-left corner */
  transform: translate(-50%, -50%); /* centers circle ON the cursor */
  pointer-events: none;           /* doesn't block clicks */
  z-index: 9999;                  /* always on top */
}
```

> ⚠ `transform: translate(-50%, -50%)` is critical. Without it, the top-left corner of the div sits on the mouse pointer instead of the center of the circle.

6.2 `animateNavbar()`

What it does: Animates navbar elements (logo, links, button) dropping from above on page load, then chains hero section animations after.

```
const tl = gsap.timeline();

tl.from('.logo', { y: -200, opacity: 0, duration: 0.5 }, 'start')
  .from('#nav-links li', { y: -200, stagger: 0.2 })    // after logo
  .from('#nav-btn', { y: -200 }, '-=0.5')              // overlaps by 0.5s
  .from('.hero p.gradient', { opacity: 0, y: -20 })
  .from('.hero h1', { rotateX: 25, rotateZ: -4 });
```

Timeline Position Parameter:

Value	Meaning
`'start'`	Starts at the same time as the `'start'` label
`'-=0.5'`	Starts 0.5s before previous animation ends (overlap)
`'+=0.3'`	Starts 0.3s after previous animation ends (gap)
No argument	Starts immediately after previous animation ends

Why `stagger: 0.2` on nav links?

Stagger adds sequential delay between elements. With `stagger: 0.2`:

- Link 1 drops at `0.0s`
- Link 2 drops at `0.2s`
- Link 3 drops at `0.4s`

This creates a cascading waterfall effect instead of all links dropping together.

Hero heading 3D rotation:

```
gsap.from('.hero h1', {
  rotateX: 25,                  // top leans away, bottom toward viewer
  rotateZ: -4,                  // slight counter-clockwise tilt
  transformPerspective: 1000,   // camera distance — lower = more dramatic
  transformOrigin: 'left center', // rotation pivots from left edge
});
```

> Using `rotateX` alone looks mechanical. Adding slight `rotateZ` (-4°) makes it feel like text is naturally falling into place — like a page turning.

6.3 `animateHeroHeading()`

What it does: Splits "Night Live 2026" into individual characters, then animates left half from left and right half from right — meeting at the center.

Step 1 — Split text into characters:

```
const textArray = headingText.split('');    // ['N','i','g','h','t',' ','L'...]
const halfLength = textArray.length / 2;   // middle index
```

Step 2 — Wrap each character in a span:

```
textArray.forEach((char, index) => {
  if (char === ' ') {
    // Space needs special treatment — plain span collapses
    newText += `<span style="display:inline-block; width:0.3em"> </span>`;
  } else if (index < halfLength) {
    newText += `<span class='left-span'>${char}</span>`;
  } else {
    newText += `<span class='right-span'>${char}</antenna>`;
  }
});
headingElement.innerHTML = newText;
```

Step 3 — Animate both halves:

```
gsap.from('#heading .left-span', {
  y: 100, opacity: 0, duration: 0.6,
  stagger: 0.1   // left to right wave
});

gsap.from('#heading .right-span', {
  y: 100, opacity: 0, duration: 0.6,
  stagger: -0.1  // negative = right to left wave (meets center)
});
```

> ⚠ `display: inline-block` on each span is **mandatory** in CSS. GSAP cannot animate `transform` (x, y, rotation) on inline elements. inline-block makes each character an independent block.

6.4 `animateAbout()`

What it does: Animates about section elements as user scrolls into view. Left content slides from left, right image slides from right, floating card bounces in, background has parallax effect.

ScrollTrigger config:

```
scrollTrigger: {
  trigger: '.about .left', // which element to watch
  start: 'top 80%',        // when element's top reaches 80% down viewport
  end: 'top 30%',          // when element's top reaches 30% down viewport
  scrub: 0.5,              // tied to scroll with 0.5s lag
}
```

Understanding `start` and `end`:

Format is: `'element-position viewport-position'`

Value	Meaning
`'top 80%'`	Element top edge at 80% down the viewport
`'top center'`	Element top edge at screen center
`'top top'`	Element top edge at screen top (used for pinning)
`'bottom top'`	Element bottom edge at screen top (fully scrolled past)

`scrub` **values:**

Value	Behavior
`scrub: true`	1:1 direct scroll tie, no smoothing
`scrub: 0.5`	Follows scroll with 0.5s lag
`scrub: 1`	Follows scroll with 1s lag (smoother)
No scrub	Plays once when triggered, not tied to scroll

Parallax background:

```
gsap.to('.bg-img-box img', {
  yPercent: 20,  // image moves 20% of its height during scroll
  ease: 'none',  // linear — constant speed (no easing for parallax)
  scrollTrigger: {
    trigger: '.about',
    start: 'top bottom',
    end: 'bottom top',
    scrub: true,
  }
});
```

> The container has `overflow: hidden`. The image inside is larger than the container. As you scroll, the image moves at a different speed — creating depth illusion.

6.5 `animateAboutImage()`

What it does: When hovering over the about section image, the custom cursor morphs from a small circle into a large rectangle showing an image inside it.

```
aboutImage.addEventListener('mouseenter', () => {
  gsap.to('#cursor', {
    width: 180,           // grows from 40px to 180px
    height: 220,          // grows from 40px to 220px
    borderRadius: '12px', // circle becomes rectangle
    duration: 0.5,
```

```
  });
  cursor.classList.add('show-img'); // fades in inner image
});

aboutImage.addEventListener('mouseleave', () => {
  gsap.to('#cursor', {
    width: 40,
    height: 40,
    borderRadius: '50%',  // becomes circle again
    duration: 0.4,
  });
  cursor.classList.remove('show-img');
});
```

The `show-img` CSS selector:

```
/* Default — image hidden */
.cursor-inner-img { opacity: 0; }

/* When class added — image visible */
.cursor-box.show-img .cursor-inner-img { opacity: 1; }
```

> ⚠ `.cursor-box.show-img` (no space) means `show-img` is ON `cursor-box` itself.
> `.cursor-box .show-img` (with space) means find `show-img` INSIDE `cursor-box` — **wrong!**

6.6 `animateArtist()`

What it does: Handles cursor hiding in artist section, scroll-triggered card entrance animations, image parallax, and 3D tilt effect on mousemove.

3D Tilt Effect:

```
card.addEventListener('mousemove', (e) => {
  const rect = card.getBoundingClientRect();

  // Mouse position as 0-1 within the card
  const x = (e.clientX - rect.left) / rect.width;
  const y = (e.clientY - rect.top) / rect.height;

  // Convert to -0.5 to +0.5 (0 at center)
  const rotateY = (x - 0.5) * 12;  // left-right, max 12°
  const rotateX = -(y - 0.5) * 12; // up-down, max 12°

  gsap.to(card, {
    rotateX, rotateY,
    scale: 1.03,
    transformPerspective: 900,
```

```
    duration: 0.4,
    ease: 'power2.out'
  });

  // Image moves OPPOSITE direction — depth illusion
  gsap.to(card.querySelector('img'), {
    x: (x - 0.5) * -15,
    y: (y - 0.5) * -15,
    duration: 0.4
  });
});
```

Why `getBoundingClientRect()`?

Returns exact position and size of element relative to viewport. Needed to calculate mouse position as 0-1 value inside the card. Without it, we can't know if mouse is at left edge or right edge.

Why image moves opposite to card?

Card tilts right → image moves left. This creates a parallax between card surface and image — like there's depth between them. Makes it feel truly 3D rather than a flat tilt.

Spring back on mouseleave:

```
gsap.to(card, {
  rotateX: 0, rotateY: 0, scale: 1,
  ease: 'elastic.out(1, 0.6)', // spring bounce effect
  duration: 0.7
});
// elastic.out(amplitude, period)
// amplitude: 1 = normal spring
// period: 0.6 = how quickly spring settles
```

6.7 `animatePlans()`

What it does: Pricing cards slide in from opposite sides when the section scrolls into view with a 3D rotation effect.

```
gsap.from('#left-plan', {
  x: -300,
  opacity: 0,
  rotateX: 45,              // tilted backward (top away from viewer)
  transformPerspective: 900,
  ease: 'power3.out',
  scrollTrigger: {
    trigger: '#plans-section',
    start: 'top 80%',
```

```
    toggleActions: 'play none none none',
  }
});
```

`toggleActions` **values:**

```
toggleActions: 'onEnter  onLeave  onEnterBack  onLeaveBack'
               'play     none     none          none'
```

Value	Action
`play`	Play animation forward
`reverse`	Play animation backward
`restart`	Start from beginning
`reset`	Jump back to start state
`none`	Do nothing

6.8 `animateGallery()`

What it does: The most complex function. Handles intro text animations and the pinned horizontal scroll gallery with individual image entrance animations.

Core horizontal scroll logic:

```
const wrapper = document.querySelector('#gallery-img-wrapper');
const totalScrollWidth = wrapper.scrollWidth - window.innerWidth;
// scrollWidth = total width of all images side by side
// window.innerWidth = visible screen width
// Difference = how far we need to scroll horizontally

const galleryTween = gsap.to(wrapper, {
  x: -totalScrollWidth,   // move left by that amount
  ease: 'none',           // linear — constant scroll feel
  scrollTrigger: {
    trigger: '.gallery-wrapper',
    start: 'top top',     // pin when wrapper hits viewport top
    end: () => `+=${totalScrollWidth + 200}`,
    pin: true,            // freeze section while horizontal scroll happens
    scrub: 1,
    anticipatePin: 1,     // prevents jump when pinning starts
  }
});
```

Why arrow function for `end`?

```
// ❌ Wrong — calculated once on page load, breaks on resize
end: `+=${totalScrollWidth}`,

// ☑ Correct — recalculated every time it's needed
end: () => `+=${totalScrollWidth}`,
```

How pin works:

`pin: true` freezes `.gallery-wrapper` in place. The page keeps scrolling but the section stays fixed. GSAP uses that scroll progress to drive the horizontal `x` movement. So vertical scroll = horizontal movement.

`containerAnimation` for individual images:

```
gsap.from(img, {
  opacity: 0, scale: 0.8,
  scrollTrigger: {
    trigger: img,
    containerAnimation: galleryTween, // watch horizontal position, not
vertical
    start: 'left 85%',                // when image left edge hits 85% of
viewport
    toggleActions: 'play none none none',
  }
});
```

Required CSS:

```
.gallery-wrapper      { overflow: hidden; }
.gallery-img-wrapper  {
  display: flex;
  flex-wrap: nowrap;       /* MUST — prevents row wrapping */
  overflow: visible;       /* NOT overflow-x: auto — GSAP handles it */
  will-change: transform;
}
.gallery-img-wrapper img {
  flex-shrink: 0;          /* MUST — prevents shrinking */
  height: 80vh;
  width: auto;
}
```

6.9 `animateTextSection()`

What it does: Creates a continuous wave animation on "LANGUAGE OF THE SOUL" text. Each character bobs up and down with slight delay from the previous.

```
// Split into character spans
textArray.forEach((char) => {
  newText += `<span>${char}</span>`;
});

// Wave animation
gsap.to('#anim-big span', {
  y: -50,
  duration: 0.6,
  ease: 'sine.inOut',  // smooth S-curve = organic wave feel
  stagger: {
    each: 0.05,        // 0.05s delay between characters
    repeat: -1,        // loops forever
    yoyo: true,        // reverses back down after going up
  },
  repeat: -1,
});
```

How stagger creates the wave:

Without stagger — all characters jump up together → looks like a block jumping.

With `stagger: 0.05`:

- Character 1 starts at `0.00s`
- Character 2 starts at `0.05s`
- Character 3 starts at `0.10s`

Each character is in a different part of its animation at any moment. This offset creates the visual wave.

Why `sine.inOut`?

Creates smooth S-curve acceleration — character starts slow, speeds up in middle, slows at peak. Matches natural wave motion — organic not mechanical.

6.10 `animateTextSection_SVG()`

What it does: Makes an SVG blob shape follow the cursor inside the text section. Hides the main cursor while inside.

```
// Hide main cursor
section.addEventListener('mouseenter', () => {
  cursor.style.display = 'none';
});
```

```
// SVG blob follows mouse
section.addEventListener('mousemove', (e) => {
  gsap.to('#blob', {
    x: e.clientX - 60,  // -60 centers 120px wide blob on cursor
    y: e.clientY - 60,
    duration: 0.3,
    ease: 'power2.out'
  });
});

// Show cursor again
section.addEventListener('mouseleave', () => {
  cursor.style.display = 'flex';
});
```

Required CSS:

```
#blob {
  position: fixed;       /* relative to viewport */
  top: 0; left: 0;       /* GSAP x/y moves from here */
  width: 120px;
  height: 120px;
  pointer-events: none;  /* doesn't block mouse events */
  z-index: 9998;
}
```

> **Why `-60` offset?** GSAP sets x/y from top-left corner. Blob is 120px wide. To center it on cursor, subtract half the width (60px) from both x and y.

6.11 `animateFooter()`

What it does: Combined function handling cursor hiding, logo hover image reveal, and all scroll-triggered entrance animations.

Logo hover image reveal:

```
logo.addEventListener('mousemove', (e) => {
  gsap.to(image, {
    // Convert viewport coords to footer-relative coords
    x: e.clientX - footer.getBoundingClientRect().left - 150,
    y: e.clientY - footer.getBoundingClientRect().top - 190,
    duration: 0.5,
  });
});
```

Why subtract `getBoundingClientRect()`?

The image is `position: absolute` inside the footer. Its x/y are relative to the footer's top-left corner, NOT the viewport. `clientX/Y` is relative to the viewport. So we subtract the footer's viewport position to convert to footer-relative coordinates.

Footer animation summary:

Element	Animation
`.footer-logo`	Slides in from left (`x: -100`)
`.footer-tagline`	Slides from left with delay
`.footer-divider`	Grows from top (`scaleY: 0 → 1`)
`.footer-nav-item`	Stagger fade from left
`.footer-newsletter`	Slides from right (`x: 80`)
`.social-icon`	Scale bounce with `back.out`
`.footer-ghost-text`	Slides from far right (`x: 200`)

📚 GSAP Core Concepts

Method	Syntax	What it does
`gsap.from()`	`gsap.from(el, { x: -100 })`	Animates FROM values TO current state
`gsap.to()`	`gsap.to(el, { x: 100 })`	Animates FROM current state TO values
`gsap.timeline()`	`const tl = gsap.timeline()`	Chain animations in sequence
`stagger`	`stagger: 0.1`	Delay between multiple elements
`yoyo`	`yoyo: true`	Reverses animation back to start
`repeat`	`repeat: -1`	Loop animation (-1 = infinite)
`scrub`	`scrub: 1`	Tie animation to scroll position
`pin`	`pin: true`	Freeze element during scroll
`ease`	`ease: 'power3.out'`	Control acceleration curve
`transformPerspective`	`transformPerspective: 900`	3D depth for rotations

Easing Quick Reference

```
power1.out    →  Gentle deceleration (subtle)
power2.out    →  Medium deceleration (common use)
```

```
power3.out     →  Strong deceleration (good for reveals)
power4.out     →  Very strong (dramatic entry)
back.out(x)    →  Overshoots then settles (x = intensity)
elastic.out    →  Spring/bounce effect
sine.inOut     →  Smooth S-curve (good for waves/loops)
none/linear    →  Constant speed (good for scroll-tied animations)
```

📍 ScrollTrigger Reference

start and end Format

```
start: 'top 80%'
        ^     ^
        |     └── viewport position (80% down from top of screen)
        └─────── element position (top edge of element)

Common element positions : top, center, bottom
Common viewport positions : top, center, bottom, 80%, 50%
```

Common Examples

Value	When triggers
`'top bottom'`	Element enters from bottom of screen
`'top center'`	Element top reaches screen middle
`'top top'`	Element top reaches screen top (pinning)
`'bottom top'`	Element fully scrolled past

toggleActions

```
toggleActions: 'onEnter  onLeave  onEnterBack  onLeaveBack'

Common presets:
  'play none none none'     → play once when entering
  'play reverse none none'  → play forward entering, reverse leaving
  'restart none none none'  → restart every time element enters
```

Debug Markers

```
scrollTrigger: {
  markers: true,  // shows green (start) and red (end) lines
```

```
}
// Remove before final deployment!
```

🐛 Common Issues & Fixes

Problem	Fix
`GSAP target null not found`	Element doesn't exist. Check ID/class spelling. Ensure JS runs after HTML loads.
Animation plays on load, not scroll	ScrollTrigger not set up. Add `scrollTrigger: {}` block with `trigger` and `start`.
Horizontal scroll doesn't work	Check `flex-wrap: nowrap` and `overflow: visible` on track. Remove `overflow-x: auto`.
3D rotation looks flat	Add `transformPerspective: 900`. Without it, 3D rotations have no depth.
Cursor not centered on mouse	Add `transform: translate(-50%, -50%)` in CSS to cursor element.
span characters not animating	Add `display: inline-block` in CSS. GSAP can't transform inline elements.
Space disappears between words	Use ` ` or `width: 0.3em` on space spans. Plain spaces in spans collapse.
Markers showing in final site	Remove `markers: true` from all ScrollTrigger objects before deployment.
Image cursor doesn't show	`.cursor-box .show-img` is wrong. Use `.cursor-box.show-img` (no space).
Gallery jumps when pinning	Add `anticipatePin: 1` to the ScrollTrigger config.

🚀 Future Improvements

Animation Enhancements

- ☐ Use GSAP SplitText plugin for proper text splitting
- ☐ Add Lenis smooth scroll for butter-smooth page scrolling
- ☐ Add page loader animation before content appears
- ☐ Add cursor trail effect — multiple dots following cursor
- ☐ Add magnetic button effect — buttons attract cursor when nearby

New Sections

- ☐ Event schedule section with animated table cells
- ☐ Countdown timer to festival date with animated digits
- ☐ Music player bar at bottom with festival audio preview
- ☐ Artist card flip on click to show bio
- ☐ Gallery lightbox on image click

Performance

- ☐ Lazy load gallery images with IntersectionObserver
- ☐ Use `will-change: transform` only on actively animating elements
- ☐ Add `prefers-reduced-motion` media query for accessibility
- ☐ Compress video for faster load times

Code Quality

- ☐ Extract all colors to CSS variables
- ☐ Add GSDevTools for animation debugging in development
- ☐ Split JS into multiple files — one per section
- ☐ Add TypeScript for type safety on animation configs

How to Run

```
# Clone the repo
git clone https://github.com/yourusername/festava-live.git

# Open index.html in browser
# No build step needed — everything runs via CDN
open index.html
```

> **Note:** The video background (`hero-section-video.mp4`) is a local asset. You need to add your own video file in the `assets/` folder or replace the `<source>` tag with an external URL.

License

This project is open source and available for educational purposes.

Built with GSAP 3 + ScrollTrigger

Made for learning · Not for redistribution